

Nama : Chita Arthalina Sianturi
NIM : 245150407111057
Kelas : Pengembangan Aplikasi Mobile - C

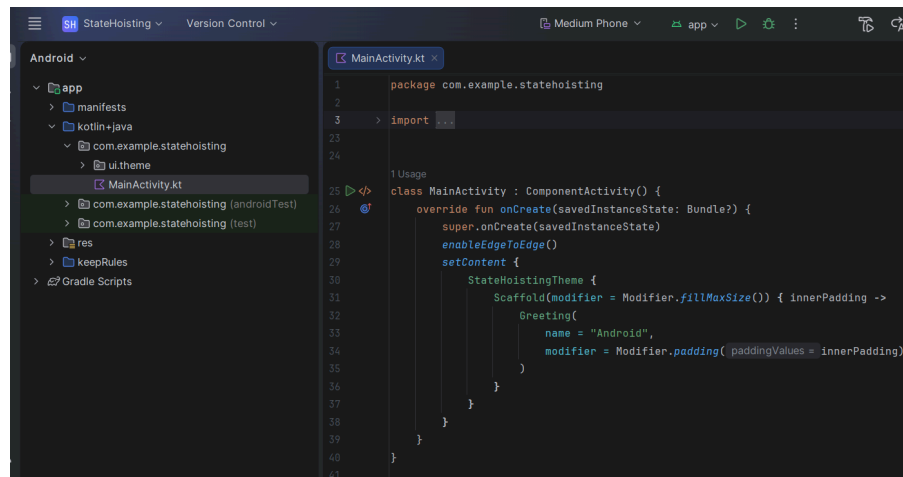
MODUL 4 PRAKTIKUM

State Hoisting, Perubahan Konfigurasi, dan Side-effect

Bagian I - State Hoisting

A. Pra-Percobaan

1. Membuat project Android Studio dengan template **Empty Activity**



Disini saya membuat project baru bernama **StateHoisting**

2. Menambahkan fungsi composable di luar class MainActivity



Selain itu, ditambahkan Import yang dibutuhkan agar tidak terjadi error.

```

1 package com.example.statehoisting
2
3 import android.os.Bundle
4 import androidx.activity.ComponentActivity
5 import androidx.activity.compose.setContent
6 import androidx.activity.enableEdgeToEdge
7 import androidx.compose.foundation.layout.fillMaxSize
8 import androidx.compose.foundation.layout.padding
9 import androidx.compose.material3.Scaffold
10 import androidx.compose.material3.Text
11 import androidx.compose.runtime.Composable
12 import androidx.compose.ui.Modifier
13 import androidx.compose.ui.tooling.preview.Preview
14 import com.example.statehoisting.ui.theme.StateHoistingTheme
15 import androidx.compose.runtime.remember
16 import androidx.compose.runtime.mutableStateOf
17 import androidx.compose.runtime.getValue
18 import androidx.compose.runtime.setValue
19 import androidx.compose.material3.Button
20 import androidx.compose.foundation.layout.Column
21 import androidx.compose.ui.unit.dp
22 import androidx.compose.ui.unit.sp

```

3. Mengubah isi fungsi onCreate()

```

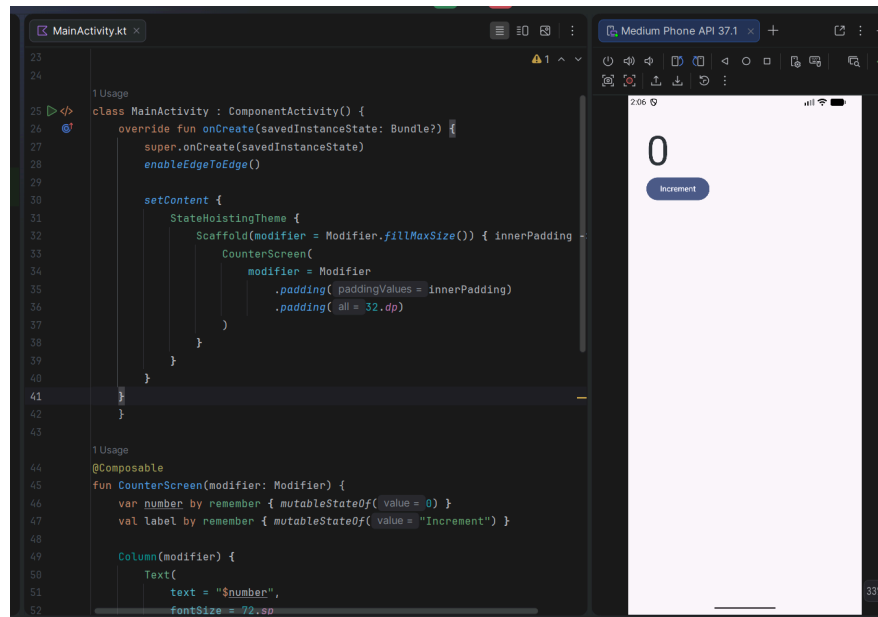
25 class MainActivity : ComponentActivity() {
26     override fun onCreate(savedInstanceState: Bundle?) {
27         super.onCreate(savedInstanceState)
28         enableEdgeToEdge()
29
30         setContent {
31             StateHoistingTheme {
32                 Scaffold(modifier = Modifier.fillMaxSize()) { innerPadding ->
33                     CounterScreen(
34                         modifier = Modifier
35                             .padding(paddingValues = innerPadding)
36                             .padding(all = 32.dp)
37                     )
38                 }
39             }
40         }
41     }
42 }

```

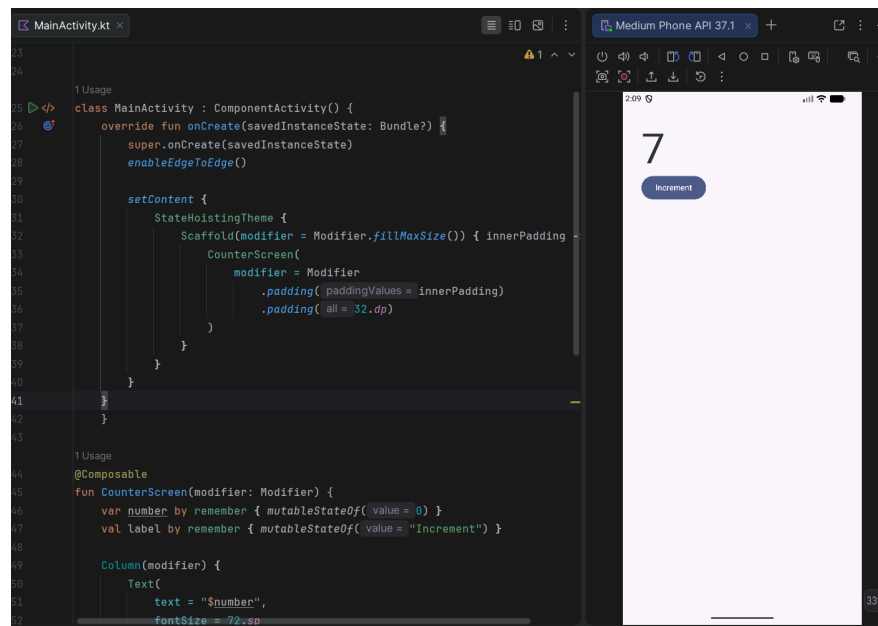
B. Aktivitas Percobaan

1. Jalankan program pada emulator. Kemudian klik tombol “Increment” beberapa kali.

Setelah program di run :



Setelah tombol increment di klik beberapa kali



- a. Apa yang terjadi setiap kali tombol "Increment" diklik?

Jawab : Setiap kali tombol **Increment** diklik, angka yang ditampilkan pada layar bertambah satu. Seperti pada gambar, nilai awal yang ditampilkan adalah 0, setelah tombol diklik satu kali nilainya menjadi 1, kemudian menjadi 2, dan seterusnya mengikuti berapa kali kita klik.

- b. Apa yang menyebabkan hal tersebut terjadi?

Jawab : Perubahan terjadi karena tombol memiliki event **onClick** yang menjalankan perintah **number++**. Perintah tersebut meningkatkan nilai variabel **number** sebesar satu setiap kali tombol ditekan karena **number** adalah state yang dikelola menggunakan **remember** dan **mutableStateOf**, perubahan nilainya menyebabkan Compose melakukan recomposition sehingga tampilan **Text** diperbarui.

- c. Variabel state apa yang berperan pada Composable Text yang menampilkan perubahan angka pada Text tersebut?

Jawab : Variabel state yang berperan adalah **number**. Variabel tersebut menyimpan nilai angka yang sedang ditampilkan. Nilainya digunakan oleh Composable Text melalui kode **text = "\$number"**, sehingga ketika nilai number berubah, teks pada layar juga diperbarui.

- d. Variabel state apa yang berperan untuk menampilkan label teks pada Composable Button yang bertuliskan "Increment" tersebut?

Jawab : Variabel state yang digunakan untuk menampilkan label pada Button adalah **label**. Variabel ini menyimpan teks "Increment" dan digunakan pada Composable Text di dalam Button melalui kode **text = "\$label"**.

2. Mengubah kode program pada Composable CounterScreen agar setiap kali tombol "Increment" diklik, nilai yang ditampilkan menjadi increment 2 (bertambah 2 setiap klik). Ubah pula label tombol "Increment" menjadi "Increment 2".

```
@Composable
fun CounterScreen(modifier: Modifier) {
    var number by remember { mutableStateOf( value = 0) }
    val label by remember { mutableStateOf( value = "Increment 2") }

    Column(modifier) {
        Text(
            text = "$number",
            fontSize = 72.sp
        )

        Button(
            onClick = { number+=2 }
        ) {
            Text(text = "$label")
        }
    }
}
```

Pada fungsi CounterScreen(), saya mengubah kode :

`val label by remember { mutableStateOf("Increment") }`, menjadi :

```
val label by remember { mutableStateOf("Increment 2") }
```

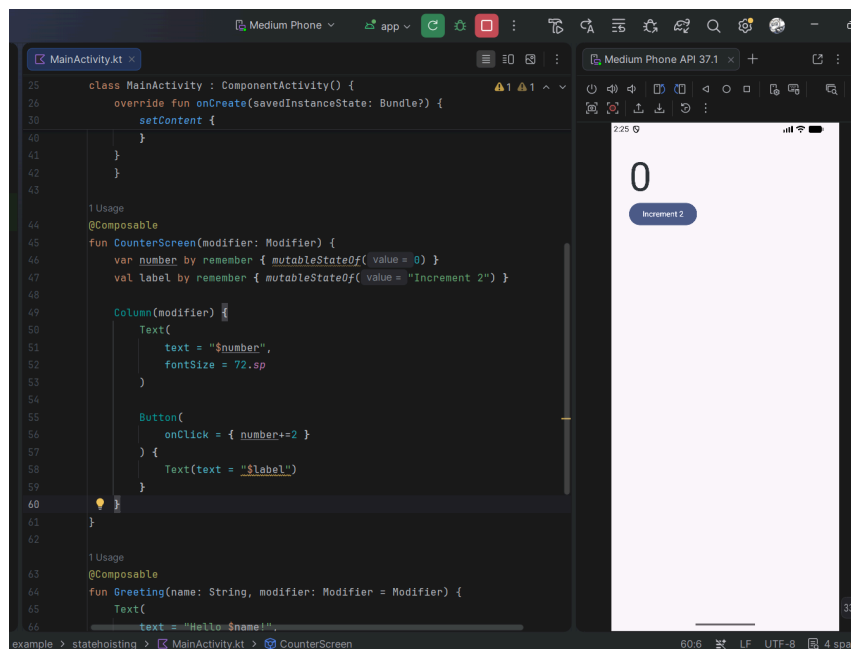
Selain itu, saya juga mengubah kode : `onClick = { number++ }`, menjadi :
`onClick = { number += 2 }`

- a. Perubahan apa yang perlu dilakukan dan dilakukan di mana?

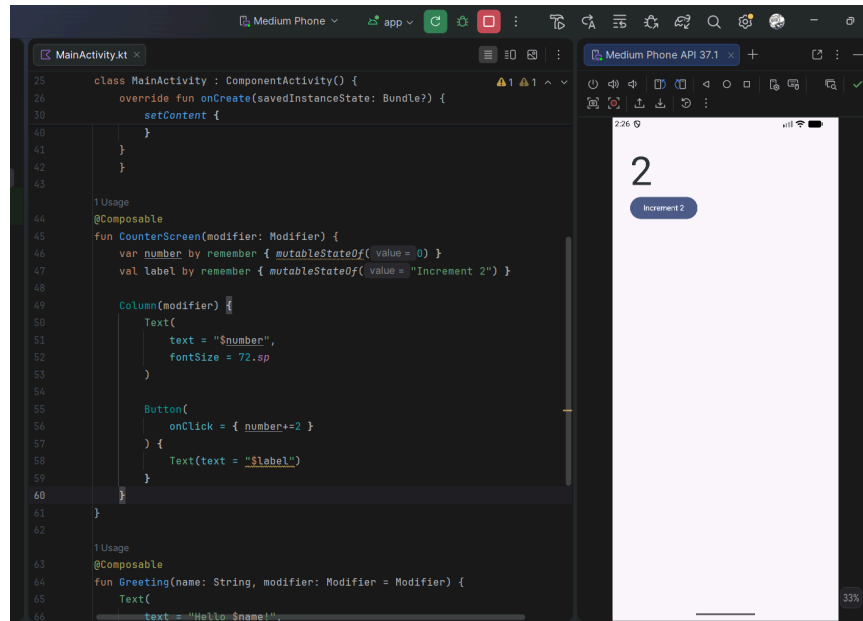
Jawab : Perubahan dilakukan pada fungsi **CounterScreen()**. Pertama, label tombol diubah dari **"Increment"** menjadi **"Increment 2"**. Kedua, operasi pada `onClick` diubah dari **`number++`** menjadi **`number += 2`**. Dengan demikian, setiap tombol ditekan nilai number akan bertambah dua dan label tombol juga menunjukkan perubahan tersebut.

- b. Jalankan program dan ambil masing-masing satu screenshot sebelum dan sesudah tombol Increment diklik untuk memperlihatkan bahwa perubahan tersebut berjalan dengan baik.

Sebelum tombol "Increment 2" diklik :



Setelah tombol "Increment 2" diklik :



3. Mengubah signature kode program pada bagian fungsi Composable CounterScreen untuk melakukan state dan event hoisting pada state dan event dalam Composable CounterScreen

```

1 Usage
@Composable
fun CounterScreen(modifier: Modifier, number: Int, label: String,
    onClick: () -> Unit) {

```

Mengubah body dalam Composable CounterScreen

```

1 Usage
@Composable
fun CounterScreen(modifier: Modifier, number: Int, label: String,
    onClick: () -> Unit) {
    // var number by remember { mutableStateOf(0) }
    // val label by remember { mutableStateOf("Increment") }
    Column(modifier) {
        Text(text = "$number", fontSize = 72.sp)
        Button(onClick = onClick) {
            Text(text = "$label")
        }
    }
}

```

Ubah pemanggilan fungsi Composable CounterScreen di dalam fungsi onCreate()

```

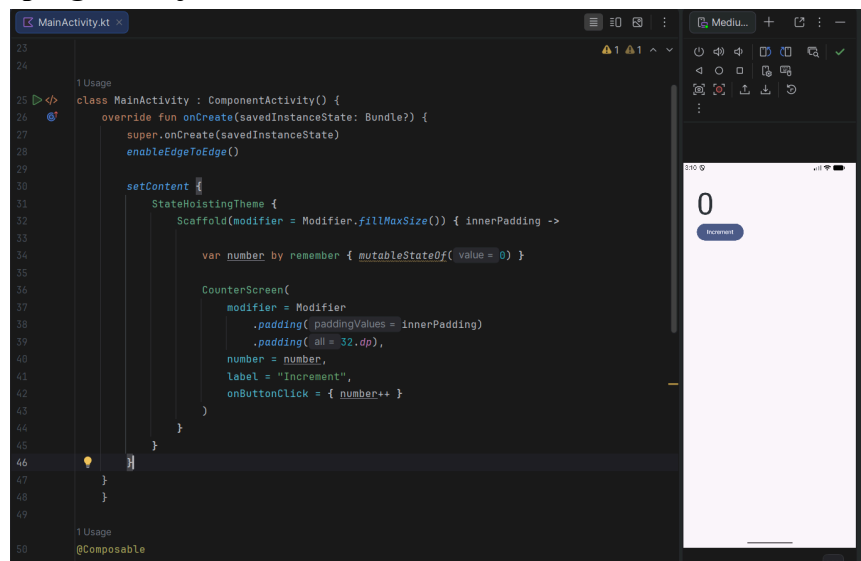
1 Usage
25 class MainActivity : ComponentActivity() {
26     override fun onCreate(savedInstanceState: Bundle?) {
27         super.onCreate(savedInstanceState)
28         enableEdgeToEdge()
29
30         setContent {
31             StateHoistingTheme {
32                 Scaffold(modifier = Modifier.fillMaxSize()) { innerPadding ->
33
34                     var number by remember { mutableStateOf( value = 0) }
35
36                     CounterScreen(
37                         modifier = Modifier
38                             .padding( paddingValues = innerPadding)
39                             .padding( all = 32.dp),
40                         number = number,
41                         label = "Increment",
42                         onClick = { number++ }
43                     )
44                 }
45             }
46         }
47     }
48 }

```

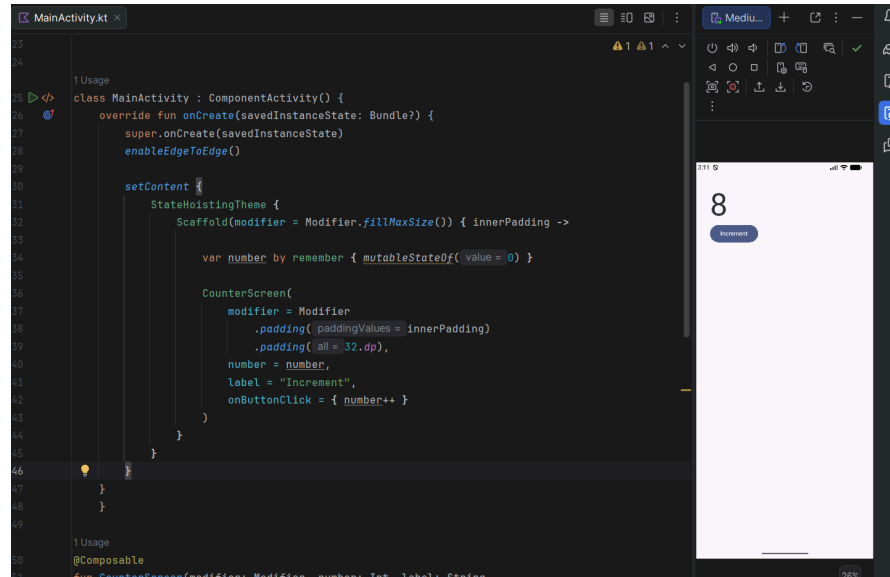
Deklarasi variabel state number dari yang sebelumnya dilakukan di dalam fungsi Composable CounterScreen sekarang dideklarasikan di dalam fungsi onCreate() sebelum fungsi Composable CounterScreen dipanggil dari dalam fungsi onCreate().

4. Jalankan program, dan kemudian klik beberapa kali tombol Increment sehingga angka yang ditampilkan berubah.

Setelah program dijalankan :



Setelah menekan tombol Increment beberapa kali :



- a. Apakah program berjalan dengan baik serta menampilkan hal yang sama persis seperti yang dilakukan pada tahap nomor 4 sebelumnya?

Jawab : Ya, program tetap berjalan dengan baik dan menghasilkan perilaku yang sama, yaitu nilai angka bertambah setiap kali tombol Increment diklik.

- b. Di bagian kode program manakah saat ini proses increment dilakukan?

Jawab : Setelah dilakukan state hoisting, proses increment dilakukan pada fungsi **onButtonClick** ketika CounterScreen() dipanggil, yaitu melalui :
lambda { number++ }

Dengan demikian, CounterScreen() hanya bertugas menampilkan nilai dan meneruskan event klik melalui parameter onButtonClick

- c. Di bagian kode program manakah saat ini state Composable Text dan Composable Button "Increment" dideklarasikan?

Jawab : Setelah dilakukan state hoisting, state number dideklarasikan di dalam fungsi onCreate() sebelum pemanggilan CounterScreen(). Sementara itu, Composable Text dan Button tetap dideklarasikan di dalam fungsi CounterScreen(). CounterScreen() menerima number dan label sebagai parameter untuk menampilkan nilai tersebut, serta menerima onButtonClick sebagai callback untuk menangani event tombol. Dengan ini, state dikelola oleh onCreate(), sementara CounterScreen() berfokus pada tampilan UI.

5. Diasumsikan terdapat perubahan kebutuhan fungsional aplikasi yang memaksa terjadinya perubahan logika berjalannya program pada antar muka pengguna. Ubah kode program setelah tahap nomor 7 dilakukan sedemikian rupa sehingga:

- a. Label tombol "Increment" menjadi "Double"

```
CounterScreen(  
    modifier = Modifier  
        .padding( paddingValues = innerPadding)  
        .padding( all = 32.dp),  
    number = number,  
    label = "Double",  
    onClick = { number++ }  
)
```

Dalam fungsi CounterScreen(), dibagian label, saya mengubah baris kode: label = "Increment" menjadi label = "Double".

- b. Nilai awal variabel number dari 0 menjadi 1

```
setContent {  
    StateHoistingTheme {  
        Scaffold(modifier = Modifier.fillMaxSize()) { innerPadding ->  
            var number by remember { mutableStateOf( value = 1) }  
        }  
    }  
}
```

Disini saya mengubah baris kode

var number by remember { mutableStateOf(0) }, menjadi :
var number by remember { mutableStateOf(1) }

- c. Ubah fungsi/lambda onClick pada pemanggilan Composable CounterScreen di dalam fungsi onCreate() sedemikian rupa sehingga yang dilakukan dari sebelumnya: number++ menjadi menggandakan nilai variabel number.

```
CounterScreen(  
    modifier = Modifier  
        .padding( paddingValues = innerPadding)  
        .padding( all = 32.dp),  
    number = number,  
    label = "Double",  
    onClick = { number*=2 }  
)
```

Pada bagian onClick dalam fungsi CounterScreen(), saya mengubah baris kode onClick = { number++ } menjadi
onClick = { number *= 2 }

- d. Tuliskan bagian kode program yang mendeklarasikan state dan pemanggilan fungsi Composable CounterScreen di dalam fungsi onCreate()

```
var number by remember { mutableStateOf(1) }  
  
CounterScreen(  
    modifier = Modifier
```

```

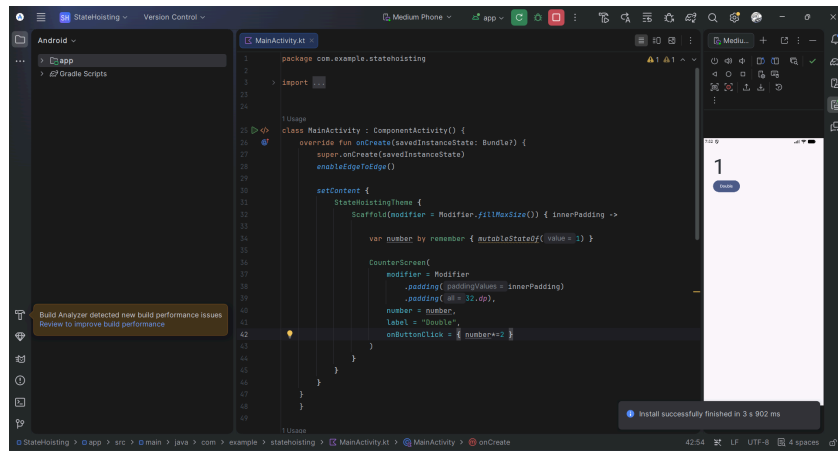
        .padding(innerPadding)
        .padding(32.dp),
        number = number,
        label = "Double",
        onClick = { number *= 2 }
    )
}

```

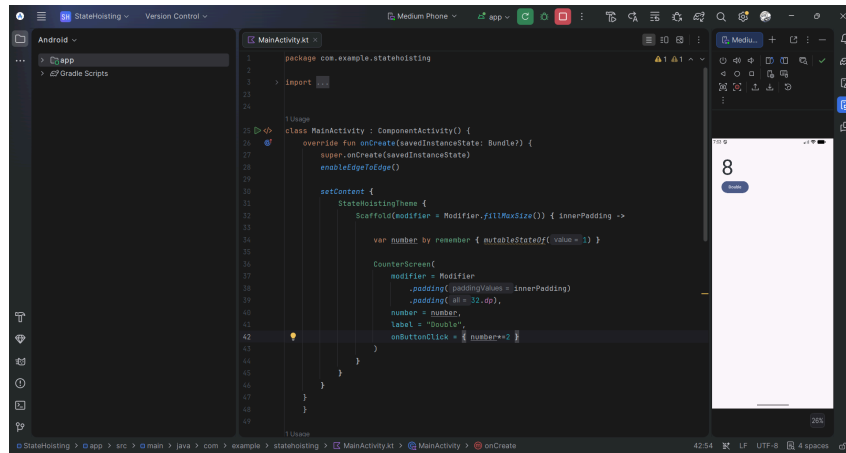
6. Jalankan program

- Perlihatkan dengan mengambil satu screenshot sebelum dan sesudah nilai variabel number digandakan.

Setelah program dijalankan :



Setelah tombol di klik beberapa kali :



Setelah tombol double diklik beberapa kali, nilai angka nya bertambah dengan kelipatan dua. Disini saya mencoba klik tombol sebanyak 3 kali dan hasil angka yang ditampilkan adalah 8 (yang semulanya adalah angka 1)

- b. Di bagian manakah dari kode program pada percobaan tahap nomor 7 dan tahap nomor 8 dilakukan?

Jawab : Perubahan pada tahap tersebut dilakukan pada fungsi **onCreate()**, khususnya pada deklarasi state **number** dan pemanggilan **CounterScreen()**. Nilai awal **number** diubah menjadi 1, **label** diubah menjadi "**Double**", dan logika pada **onButtonClick** diubah menjadi **number *= 2**.

- c. Apakah ada perubahan yang dilakukan dari tahap nomor 7 dan tahap nomor 8 yang dilakukan di dalam fungsi **Composable CounterScreen** itu sendiri? Jika ada sebutkan perubahan yang dilakukan.

Jawab : Tidak terdapat perubahan pada fungsi **CounterScreen()** itu sendiri. Hal ini menunjukkan manfaat state hoisting karena perubahan logika aplikasi dapat dilakukan pada bagian pemanggil atau pengelola state tanpa perlu mengubah komponen UI yang bersifat stateless.

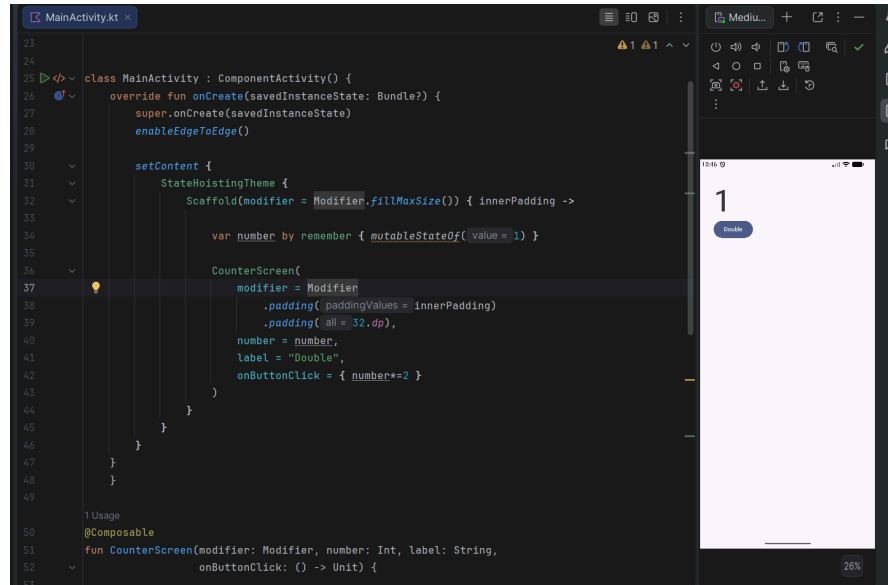
7. Apa arti dari jawaban pada tahap nomor 9 terkait dengan manfaat dari melakukan state hoisting dalam perubahan kebutuhan fungsional aplikasi yang terkait dengan antar muka pengguna?

Jawab : Dengan melakukan state hoisting, perubahan kebutuhan fungsional dipermudah karena logika pengelolaan state dipisahkan dari komponen UI. Pada percobaan, perubahan dari fungsi **increment** menjadi **double** hanya perlu dilakukan pada state dan callback di **onCreate()**, sedangkan **CounterScreen()** tetap digunakan tanpa perubahan. Dengan demikian, komponen menjadi lebih reusable, lebih mudah dipelihara, dan memiliki pemisahan yang lebih jelas antara logika UI dan pengelolaan state. Pendekatan ini juga menerapkan prinsip Single Source of Truth dan alur data satu arah.

Bagian II - Perubahan Konfigurasi

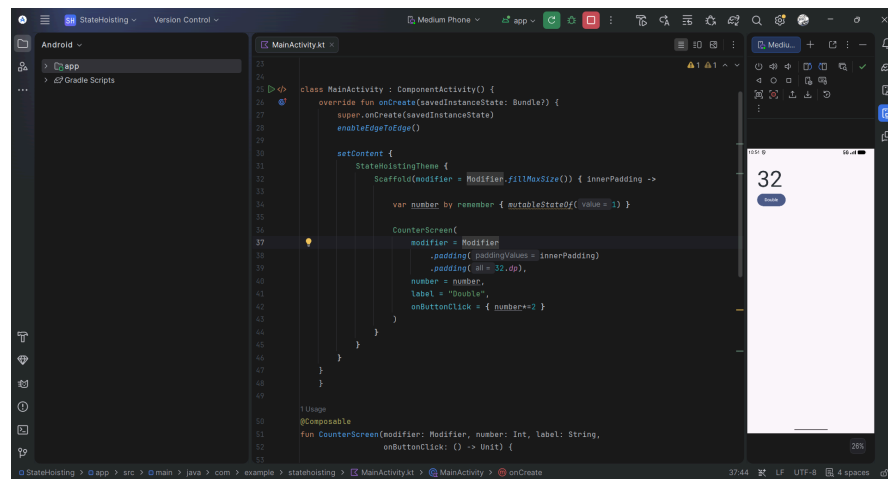
A. Pra-Percobaan

1. Menggunakan Project Android Studio yang sama dengan kode program percobaan State Hoisting dan memastikan program dapat berjalan dengan baik dan fungsionalitas aplikasi berjalan sesuai dengan fungsi yang didefinisikan (yaitu, menggandakan nilai variabel **number**).



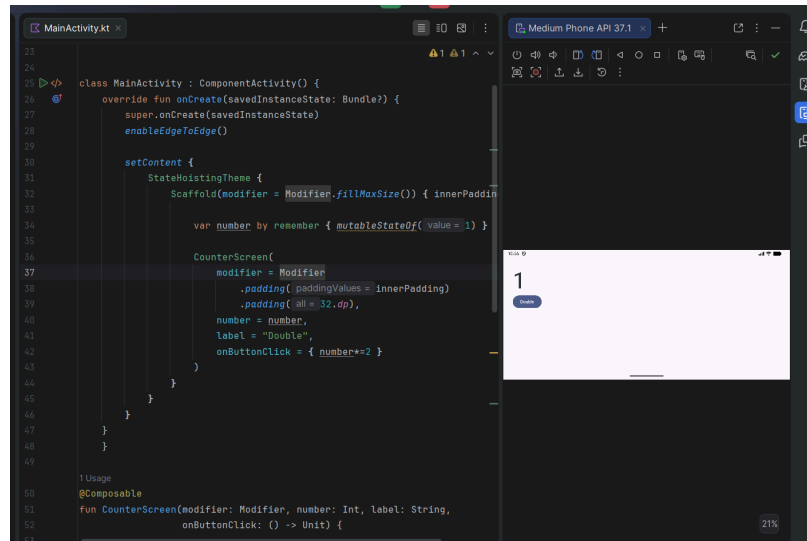
B. Aktivitas Percobaan

1. Jalankan program dan kemudian klik beberapa kali tombol "Double". Catat angka yang ditampilkan setelah tombol "Double" diklik beberapa kali, ambil screenshot tampilan antarmukanya.



Disini saya mengklik tombol “Double” sebanyak 5 kali dan menghasilkan angka 32.

2. Lakukan perubahan konfigurasi dengan memutar posisi perangkat dari posisi portrait menjadi landscape, sehingga tampilan aplikasi akan disesuaikan oleh Android.
 - a. Catat angka yang ditampilkan kemudian ambil screenshot tampilan antar muka aplikasi di posisi landscape



Angka yang ditampilkan : 1

Setelah diubah menjadi landscape, angka yang semula adalah 32 berubah menjadi angka 1.

- b. Berapa angka yang ditampilkan? Mengapa terjadi demikian?

Jawab : Setelah diubah ke posisi landscape, angka yang ditampilkan kembali menjadi 1. Hal tersebut terjadi karena rotasi perangkat merupakan perubahan konfigurasi yang menyebabkan Activity dibuat ulang. State yang disimpan menggunakan **remember** hanya dipertahankan selama proses recomposition dan tidak dirancang untuk mempertahankan state ketika Activity dibuat ulang akibat perubahan konfigurasi.

Mengubah kode program dalam fungsi onCreate(), yang sebelumnya :

```
var number by remember { mutableStateOf(1) }
```

Menjadi :

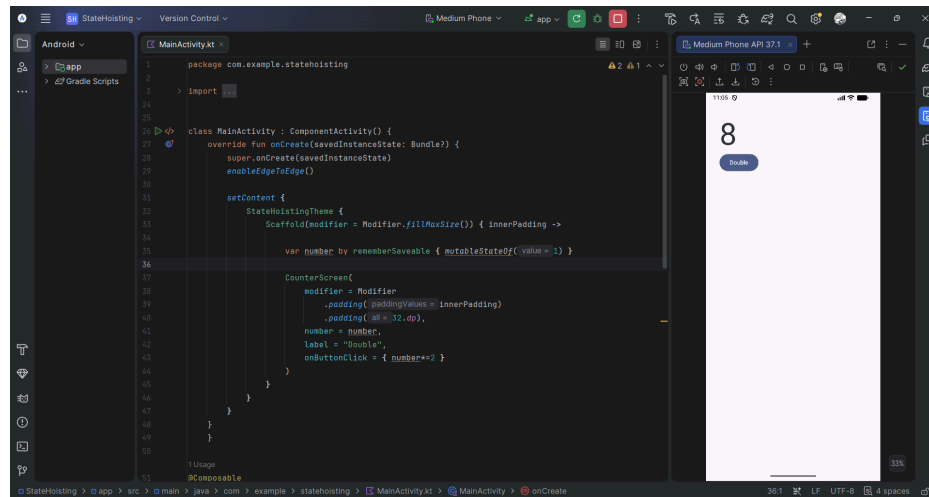
```
var number by rememberSaveable { mutableStateOf(1) }
```

Kemudian melakukan import dependency atas perubahan konstruk remember menjadi rememberSaveable yang dilakukan.

```
<> class MainActivity : AppCompatActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        enableEdgeToEdge()

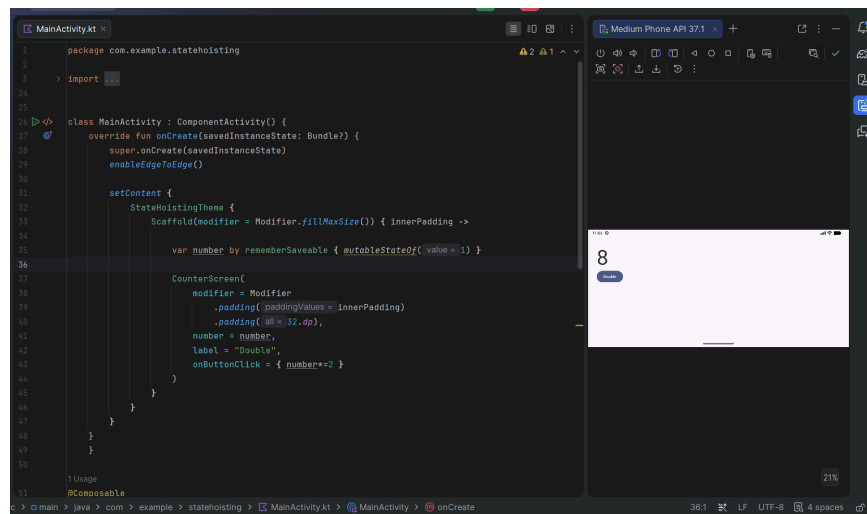
        setContent {
            StateHoistingTheme {
                Scaffold(modifier = Modifier.fillMaxSize()) { innerPadding ->
                    var number by rememberSaveable { mutableStateOf( value = 1) }
                }
            }
        }
    }
}
```

Kemudian program dijalankan kembali dengan posisi perangkat potrait, dan menekan tombol “Double” beberapa kali.



Pada percobaan saya mengklik tombol sebanyak tiga kali dan menghasilkan angka 8.

3. Lakukan kembali perubahan konfigurasi dengan memutar posisi perangkat dari posisi portrait menjadi landscape, sehingga tampilan aplikasi akan disesuaikan oleh Android.
 - a. Catat angka yang ditampilkan kemudian ambil screenshot tampilan antar muka aplikasi di posisi landscape.



Jawab : Angka yang ditampilkan setelah perangkat diubah posisi menjadi landscape adalah tetap angka 8.

- b. Berapa angka yang ditampilkan? Mengapa terjadi demikian?

Jawab : Angka yang ditampilkan setelah perangkat diputar ke posisi landscape tetap sama dengan nilai sebelum perubahan konfigurasi yaitu 8.

Hal ini terjadi karena **rememberSaveable** menyimpan state menggunakan mekanisme saved instance state sehingga nilai dapat dipertahankan ketika Activity dibuat ulang akibat perubahan konfigurasi seperti rotasi layar.

- c. Apa fungsi dari konstruk **rememberSaveable** pada kode program Android yang membedakannya dari konstruk **remember**?

Jawab : rememberSaveable berfungsi untuk mempertahankan nilai state tidak hanya selama proses recomposition, tetapi juga ketika terjadi pembuatan ulang Activity akibat perubahan konfigurasi, seperti rotasi layar. Berbeda dengan **remember**, yang hanya mempertahankan nilai state selama proses recomposition dan tidak mempertahankannya ketika Activity dibuat ulang akibat perubahan konfigurasi, **rememberSaveable** dapat menyimpan nilai state melalui mekanisme Bundle. Oleh karena itu, ketika perangkat diputar dari portrait ke landscape, state yang menggunakan **remember** dapat kembali ke nilai awal, sedangkan state yang menggunakan **rememberSaveable** tetap mempertahankan nilai sebelumnya.

4. Beri contoh kasus nyata lain di mana penggunaan konstruk **rememberSaveable** ini tepat digunakan dalam aplikasi Android serta sebutkan manfaat yang diperoleh dari penerapannya!

Jawab : Salah satu contoh penggunaan **rememberSaveable** adalah pada formulir pendaftaran akun yang memiliki beberapa input, seperti nama, email, dan nomor telepon. Apabila pengguna sedang mengisi formulir kemudian memutar perangkat dari portrait ke landscape, nilai yang telah dimasukkan sebaiknya tetap tersedia. Dengan **rememberSaveable**, nilai state pada input dapat dipertahankan ketika Activity dibuat ulang akibat perubahan konfigurasi. Manfaatnya adalah pengguna tidak perlu memasukkan kembali data yang sebelumnya telah diketik sehingga pengalaman penggunaan aplikasi menjadi lebih nyaman.

Bagian III - Launched Effect

A. Pra-Percobaan 1

1. Di dalam `onCreate()`, pada lambda `Theme {}`, sebelum `Scaffold`, ditambahkan deklarasi `host state` untuk menyimpan `state composable Snackbar`.

```

1 Usage
class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        enableEdgeToEdge()

        setContent {
            StateHoistingTheme {
                val snackbarHostState = remember {
                    SnackbarHostState()
                }
            }
        }
    }
}

```

2. Kemudian pemanggilan fungsi Composable Scaffold di dalam fungsi onCreate() yang sebelumnya Scaffold(modifier = Modifier.fillMaxSize()) menjadi:

```

Scaffold(
    modifier = Modifier.fillMaxSize(),
    snackbarHost = { SnackbarHost(hostState = snackbarHostState) }
)

```

```

<
    <
        <
            setContent {
                StateHoistingTheme {
                    val snackbarHostState = remember {
                        SnackbarHostState()
                    }
                }

                Scaffold(
                    modifier = Modifier.fillMaxSize(),
                    snackbarHost = { SnackbarHost(hostState = snackbarHostState) }
                ) { innerPadding ->
                    var number by rememberSaveable { mutableStateOf(value = 1) }
                }
            }
        <
    <

```

3. Menerapkan fitur Side-effect dengan LaunchedEffect.

```

LaunchedEffect(number) {
    snackbarHostState.showSnackbar("Number $number is shown!")
}

```

```

class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        setContent {
            StateHoistingTheme {

            }

            Scaffold(
                modifier = Modifier.fillMaxSize(),
                snackbarHost = { SnackbarHost(hostState = snackbarHostState) }
            ) { innerPadding ->
                var number by rememberSaveable { mutableStateOf(value = 1) }

                CounterScreen(
                    modifier = Modifier
                        .padding(paddingValues = innerPadding)
                        .padding(all = 32.dp),
                    number = number,
                    label = "Double",
                    onClick = { number *= 2 }
                )

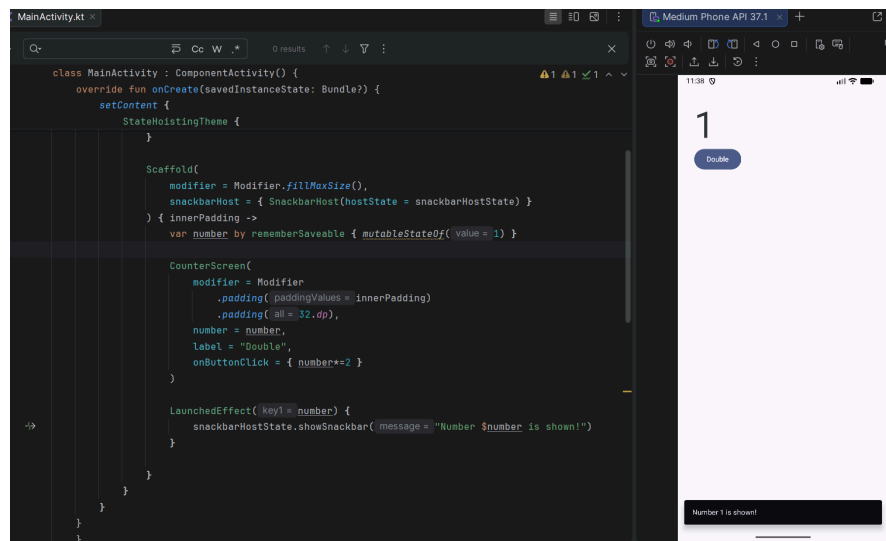
                LaunchedEffect(key1 = number) {
                    snackbarHostState.showSnackbar(message = "Number $number is shown!")
                }
            }
        }
    }
}

```

B. Aktivitas Percobaan

1. Jalankan program

- a. Pada saat pertama kali program dijalankan, tanpa mengklik button "Double" apa yang ditampilkan di dalam layar?



Jawab : Pada saat pertama kali aplikasi dijalankan, nilai number ditampilkan pada antarmuka perangkat adalah nilai awal 1. Selain itu, muncul Snackbar yang menampilkan pesan "**Number 1 is shown!**".

- b. Mengapa pesan tersebut ditampilkan saat pertama kali aplikasi dijalankan tanpa ada interaksi apa pun dari pengguna?

Jawab : Snackbar dapat muncul tanpa adanya interaksi pengguna karena **LaunchedEffect(number)** dijalankan ketika Composable pertama kali

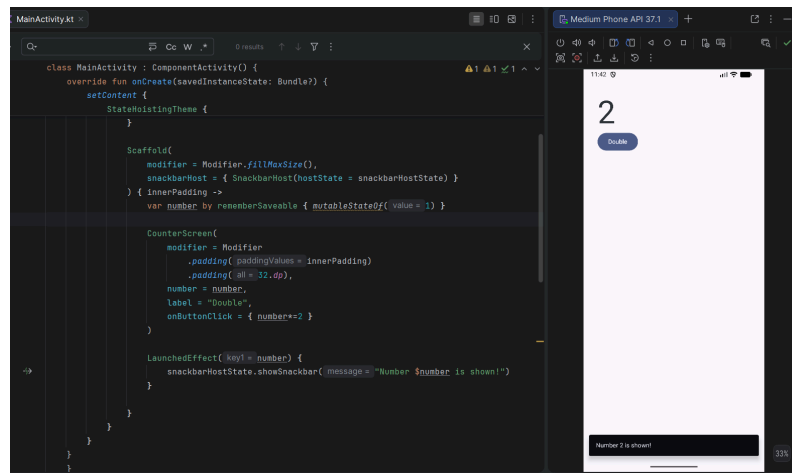
masuk ke dalam composition. Nilai number yang digunakan sebagai key pada **LaunchedEffect** memiliki nilai awal 1, sehingga effect dijalankan dan memanggil **showSnackBar()** secara otomatis.

- c. Apa yang memicu pesan pada SnackBar tersebut ditampilkan? Pada bagian kode program yang mana?

Jawab : SnackBar dipicu oleh **LaunchedEffect(number)**. Ketika Composable pertama kali masuk ke dalam composition atau ketika nilai **number** berubah, blok **LaunchedEffect** dijalankan. Di dalam blok tersebut terdapat perintah `snackbarHostState.showSnackBar("Number $number is shown!")` yang membuat pesan ditampilkan.

2. Klik button "Double" satu kali

- a. Apa yang terjadi?



Jawab : Setelah tombol Double diklik, nilai number berubah dari 1 menjadi 2. Perubahan tersebut menyebabkan **LaunchedEffect(number)** dijalankan kembali sehingga SnackBar menampilkan pesan "**Number 2 is shown!**".

- b. Mengapa hal tersebut terjadi? Jelaskan!

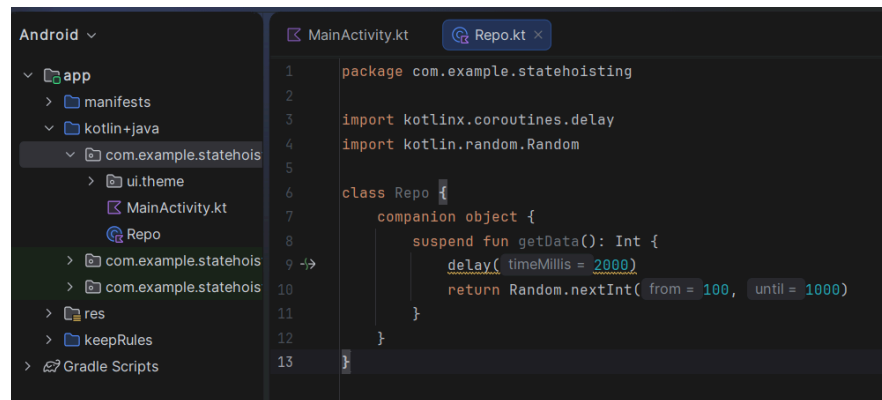
Jawab : Hal tersebut terjadi karena number digunakan sebagai key pada **LaunchedEffect**. Ketika nilai key berubah, efek yang sedang terkait dengan key sebelumnya akan dibatalkan dan dijalankan kembali menggunakan nilai key yang baru. Oleh karena itu, perubahan nilai number dari 1 menjadi 2 menyebabkan SnackBar menampilkan pesan yang sesuai dengan nilai terbaru.

C. Pra-Percobaan 2

1. Membuat class Kotlin baru bernama Repo.kt dengan struktur class dan fungsi static `getData()` dengan kode :

```
class Repo {  
    companion object {  
        suspend fun getData() : Int {  
            delay(2000) // simulating network  
            return Random.nextInt(100, 1000)  
        }  
    }  
}
```

Dan melakukan import dependency untuk fungsi `delay()` dan class `Random`.



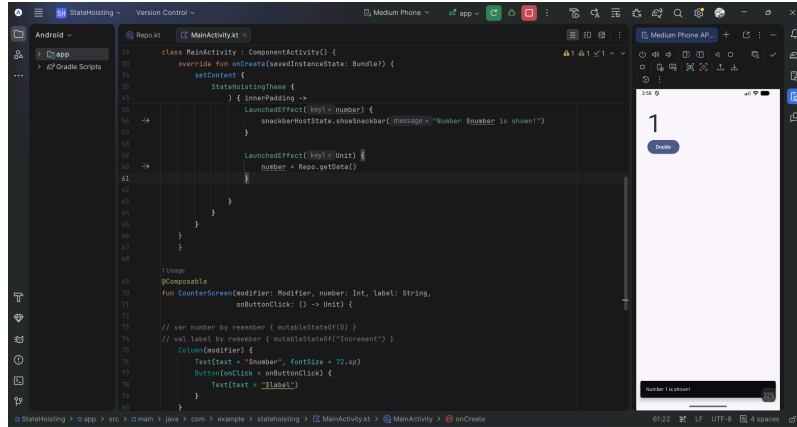
2. Di dalam fungsi `onCreate()`, setelah `LaunchedEffect` pada percobaan sebelumnya, tambahkan deklarasi fungsi `LaunchedEffect` baru

```
LaunchedEffect(Unit) {  
    number = Repo.getData()  
}
```



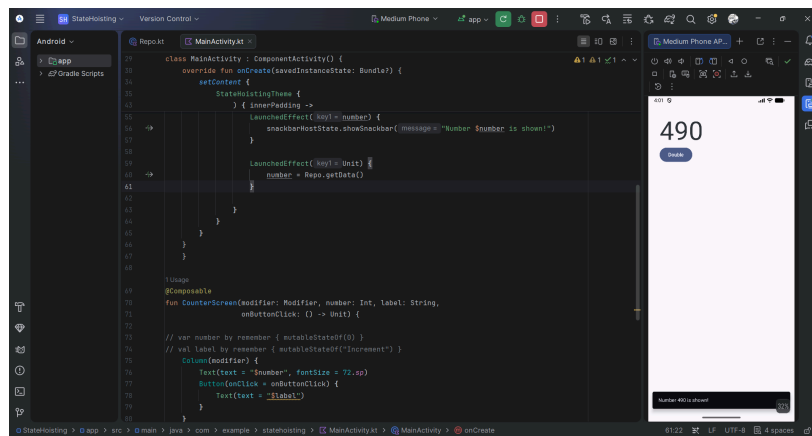
D. Aktivitas Percobaan

1. Jalankan program
 - a. Pada saat pertama kali program dijalankan, tanpa mengklik button "Double" apa yang ditampilkan di dalam layar?



Jawab : Saat pertama kali aplikasi dijalankan, perangkat menampilkan angka 1 dan juga snackbar yang menampilkan pesan "**Number 1 is shown!**".

- b. Tunggu selama dua detik, kemudian periksa kembali apa yang ditampilkan pada layar! Apa yang tampil?



Jawab : Setelah menunggu sekitar dua detik, nilai number berubah menjadi angka acak yang berada pada rentang 100 sampai 999. Pada percobaan ini, muncul angka 490 dan diikuti dengan adanya snackbar. Perubahan tersebut berasal dari fungsi **Repo.getData()** yang menggunakan **delay(2000)** untuk mensimulasikan proses pengambilan data melalui jaringan dan kemudian menghasilkan nilai menggunakan **Random.nextInt(100, 1000)**.

- c. Apa yang memicu perubahan angka dan pesan yang ditampilkan pada SnackBar tersebut ditampilkan? Pada bagian kode program yang mana?

Jawab : Perubahan angka dipicu oleh **LaunchedEffect(Unit)** yang memanggil **Repo.getData()**. Setelah proses tersebut selesai, hasilnya diberikan kepada variabel number. Perubahan nilai number kemudian

menjadi pemicu bagi **LaunchedEffect(number)**, sehingga effect tersebut dijalankan kembali dan menampilkan Snackbar dengan nilai number yang terbaru. Hal ini terjadi pada kode bagian :

```
LaunchedEffect(number) {  
    snackbarHostState.showSnackbar("Number $number is shown!")  
}
```

2. Dari kedua percobaan terkait dengan side-effect dengan LaunchedEffect

- a. Jelaskan apa perbedaan antara LaunchedEffect(state) dengan LaunchedEffect(Unit)?

Jawab : Perbedaan **LaunchedEffect(state)** dan **LaunchedEffect(Unit)** terletak pada key yang digunakan sebagai pemicu dijalankannya effect. Pada **LaunchedEffect(state)**, effect menggunakan nilai **state** sebagai key, sehingga effect akan dijalankan kembali ketika nilai state tersebut berubah. Sedangkan pada **LaunchedEffect(Unit)**, effect menggunakan **Unit** sebagai key sehingga effect tidak bergantung pada perubahan nilai state tertentu dan digunakan untuk menjalankan suatu operasi ketika composable pertama kali masuk ke composition.

- b. Kapan operasi di dalam LaunchedEffect(state) dengan LaunchedEffect(Unit) dijalankan?

Jawab : Pada **LaunchedEffect(state)**, operasi di dalamnya dijalankan ketika composable masuk ke composition dan akan dijalankan kembali setiap kali nilai **state** yang menjadi key mengalami perubahan. Contohnya pada percobaan adalah ketika tombol Double ditekan dan nilai number berubah, maka LaunchedEffect(number) akan dijalankan kembali sehingga Snackbar menampilkan angka terbaru. Sementara itu, pada **LaunchedEffect(Unit)**, operasi dijalankan ketika composable masuk ke composition dan pada percobaan ini digunakan untuk mengambil data dari Repo. Dikarenakan key-nya adalah **Unit**, perubahan nilai number tidak menjadi pemicu untuk menjalankan kembali effect tersebut.

Bagian IV - Tugas Praktik Membuat Halaman Pemesanan Tiket

A. Hasil Halaman Pesan Tiket

The image displays four sequential screens of a mobile application for ticket booking, each titled "Pemesanan Tiket".

- Screen 1 (Top Left):** The initial booking form. It includes a "Nama Pembeli" field with the placeholder "Masukkan nama Anda", a "Harga Tiket" of "Rp50.000", and a "Jumlah Tiket" of "1" with minus and plus buttons. The "Total Harga" is "Rp50.000". A blue "Pesan Tiket" button is at the bottom, and the status is "Status: Silakan pesan tiket".
- Screen 2 (Top Right):** The form after the user has entered "Chita Arthalina" and increased the quantity to "4". The "Total Harga" is now "Rp200.000". The "Pesan Tiket" button is disabled and greyed out, replaced by a "Memproses..." button. The status is "Status: Memproses pesanan...".
- Screen 3 (Bottom Left):** The confirmation screen. The "Nama Pembeli" field now contains "Chita Arthalina". The "Total Harga" remains "Rp200.000". The "Pesan Tiket" button is active (blue). The status is "Status: Tiket telah dipesan".
- Screen 4 (Bottom Right):** An error handling screen. The "Nama Pembeli" field is empty with the placeholder "Masukkan nama Anda". The "Total Harga" is "Rp200.000". The "Pesan Tiket" button is disabled. The status is "Status: Nama masih kosong".

B. Penjelasan Halaman Pesan Tiket

Halaman pemesanan tiket dibuat menggunakan Jetpack Compose dengan beberapa informasi, yaitu **nama pembeli**, **harga tiket**, **jumlah tiket**, **total harga**, dan **status pemesanan**. Harga tiket ditetapkan sebesar Rp50.000 per tiket, sedangkan jumlah tiket memiliki nilai awal 1 dan dapat diubah menggunakan tombol tambah (+) dan kurang (-). State untuk harga tiket, jumlah tiket, dan nama pembeli dikelola pada fungsi parent **TicketScreen()** menggunakan **rememberSaveable**. Nilai **ticketCount** kemudian dikirimkan ke **TicketContent()** sebagai parameter sehingga tampilan dapat menampilkan jumlah tiket yang sedang dipilih.

Perubahan jumlah tiket dilakukan melalui event **onIncrease** dan **onDecrease** yang dikelola oleh parent. Ketika tombol + ditekan, nilai **ticketCount** bertambah satu melalui **ticketCount++**, sedangkan tombol - mengurangi jumlah tiket selama nilainya masih lebih dari satu. Karena **ticketCount** merupakan state, perubahan nilainya akan menyebabkan Compose melakukan recomposition pada bagian UI yang menggunakan state tersebut. Hal yang sama juga diterapkan pada nama pembeli, di mana perubahan teks pada **OutlinedTextField** diteruskan melalui **onBuyerNameChange** sehingga nilai **buyerName** tetap dikelola oleh parent. Hal ini menunjukkan penerapan State Hoisting, yaitu state dikelola oleh parent sedangkan child menerima state dan event melalui parameter.

Untuk proses pemesanan, digunakan **LaunchedEffect** dengan **isProcessing** sebagai **key**. Ketika pengguna menekan tombol Pesan Tiket dan nama pembeli sudah diisi, nilai **isProcessing** diubah menjadi true dan status berubah menjadi “**Memproses pesanan...**”. Selanjutnya, **LaunchedEffect** menjalankan **delay(5000)** untuk memberi jeda selama lima detik. Setelah proses selesai, status diubah menjadi “**Tiket telah dipesan**” dan **isProcessing** dikembalikan menjadi false. Jika nama pembeli masih kosong, proses tersebut tidak dijalankan dan status menampilkan “**Nama masih kosong**”. Hal ini menunjukkan penerapan side-effect yang dijalankan berdasarkan perubahan key yang digunakan.